CS6650 HW6

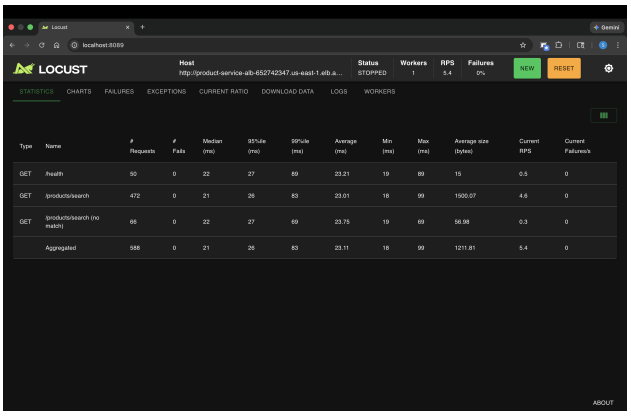
Sai Karthikeyan Sura
February 22nd, 2026

Part II: Identifying Performance Bottlenecks

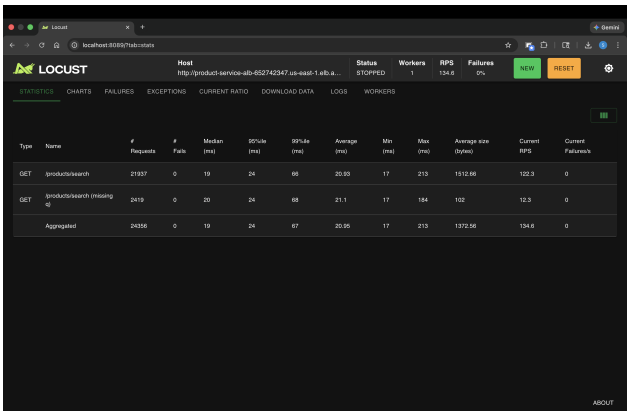
The product search service stores 100,000 products in a `sync.Map` at startup. Each search request checks 100 products starting at a random offset, performs case-insensitive matching on name and category fields, and returns at most 20 results. The service was deployed on ECS Fargate with 256 CPU units (0.25 vCPU) and 512 MB memory.

Test	Users	Requests	RPS	Median	p95	p99	Max
Baseline (256 CPU)	5	588	5.4	21ms	26ms	83ms	99ms
Breaking Point (256 CPU)	20	24,356	134.6	19ms	24ms	67ms	213ms
Breaking Point (512 CPU)	20	24,247	135.3	20ms	34ms	58ms	262ms

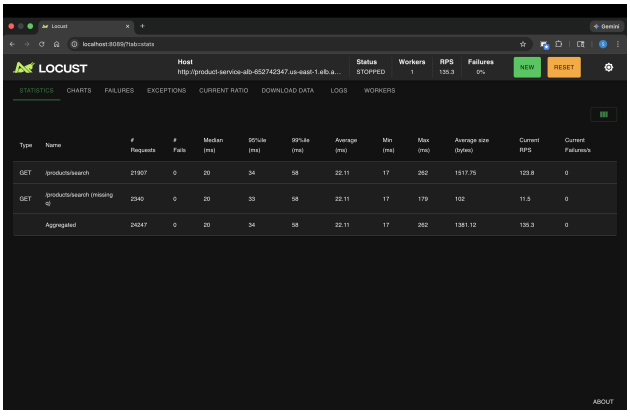
Table 1: Load test statistics across single-instance configurations.



(a) Baseline (256 CPU): 5 users



(b) Breaking Point (256 CPU): 20 users



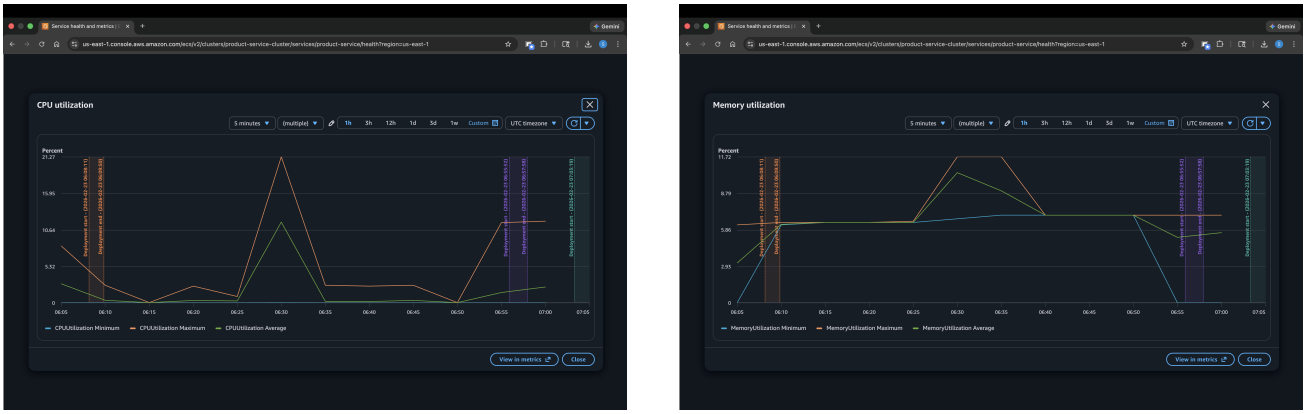
(c) Breaking Point (512 CPU): 20 users

Figure 1: Single-instance load test comparison baseline, 256-CPU and 512-CPU breaking points.

CloudWatch Metrics

Figures 2 shows CPU and memory utilization across sequential load tests. The baseline test with 5 users produced minimal CPU load, while the 20-user breaking point test raised CPU to approx-

imately 21% peak on the 256 CPU instance. This relatively low utilization despite 134.6 RPS indicates the bounded search (100 products, simple string matching) is computationally inexpensive — latency is dominated by network round-trip time, not server-side computation. Repeating the test on 512 CPU showed further reduced utilization.



(a) CPU utilization (b) Memory utilization

Figure 2: CloudWatch resource utilization.

Resource limit: CPU is the constraining resource. At 20 users, CPU reached 21% on a single 256 CPU instance, while memory stayed at 6–11%. CPU increased with load, confirming the workload is compute-bound; higher concurrency or heavier requests would push CPU toward saturation.

Response time degradation: Scaling from 5 to 20 users, median response times remained 19–21 ms and RPS reached 134.6 with 0% failures. Tail latencies increased: p99 rose to 67 ms, max to 213 ms (vs. 83 ms and 99 ms at 5 users), showing queuing under higher concurrency without full CPU saturation.

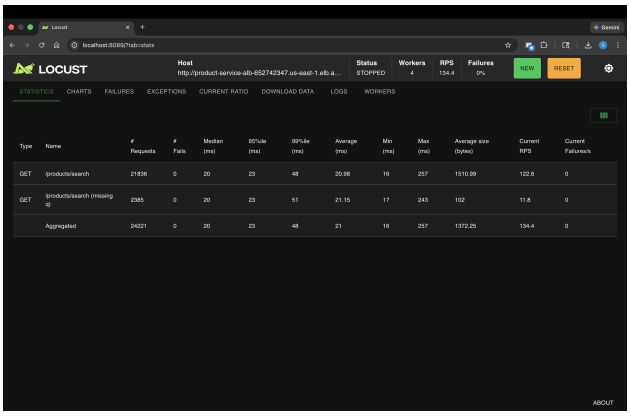
Effect of doubling CPU (256 → 512): Peak CPU utilization dropped, RPS was similar at 135.3, and p99 improved to 58 ms. Vertical scaling eases compute pressure but gives diminishing returns; sustained throughput gains require horizontal scaling across multiple instances.

Part III: Horizontal Scaling with Auto Scaling

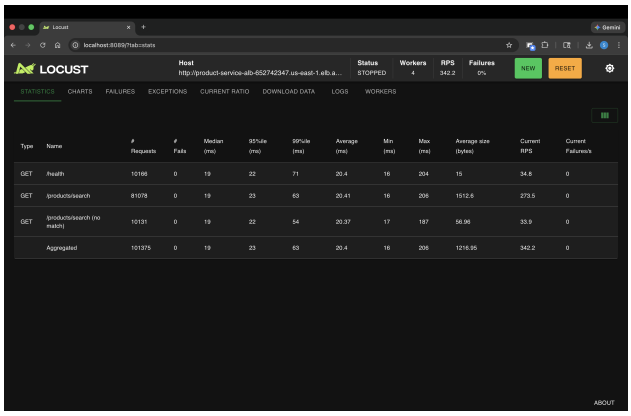
In Part II, a single 256 CPU instance showed rising CPU and tail latency under 20 users, defining the performance ceiling. Horizontal scaling distributes requests across multiple instances behind an ALB, with auto scaling (70% CPU target, 2–4 instances) adjusting capacity. As Table 2 shows, the same 20-user load now has lower tail latency (p99: 48 ms vs. 67 ms in Table 1), and the system handles 100 users at 1,269 RPS with 0% failures.

Test	Users	Requests	RPS	Median	p95	p99	Max
Breaking (2 inst)	20	24,221	134.4	20ms	23ms	48ms	257ms
Stress (2–4 inst)	50	101,375	342.2	19ms	23ms	63ms	206ms
Spike (2–4 inst)	50	115,510	644.6	20ms	25ms	85ms	398ms
100 Users (2–4 inst)	100	225,709	1269.6	19ms	46ms	120ms	316ms
Resilience (1 task down)	20	68,468	134.9	20ms	24ms	37ms	200ms
Resilience (2 tasks down)	50	62,666	339.9	19ms	22ms	62ms	205ms
50% CPU Target	20	101,344	333.3	20ms	25ms	78ms	287ms

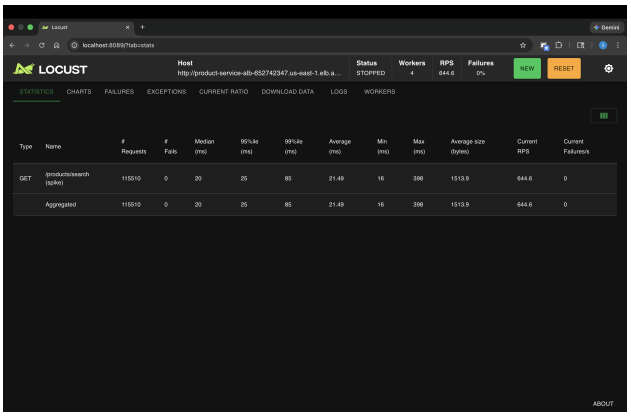
Table 2: Horizontal scaling load test statistics.



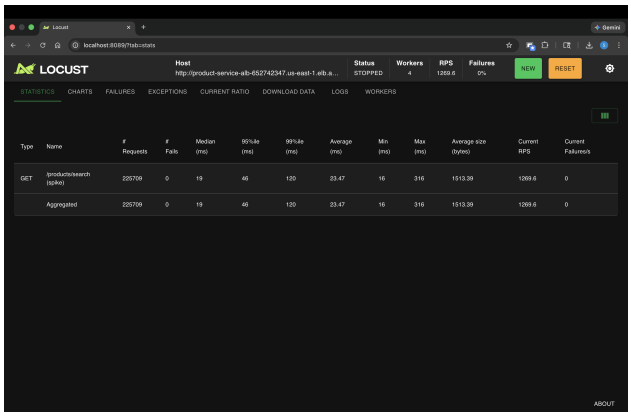
(a) Breaking (2 instances): 20 users



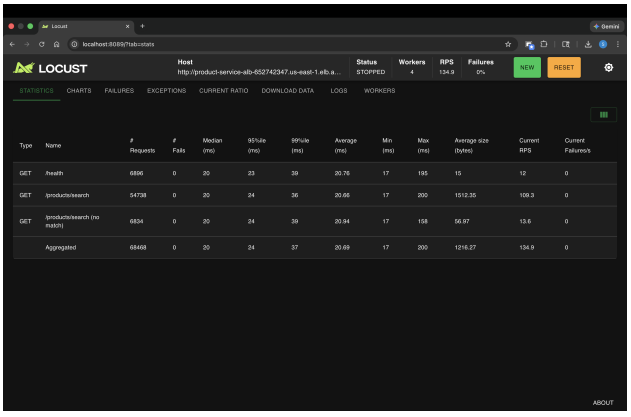
(b) Stress (2–4 instances): 50 users



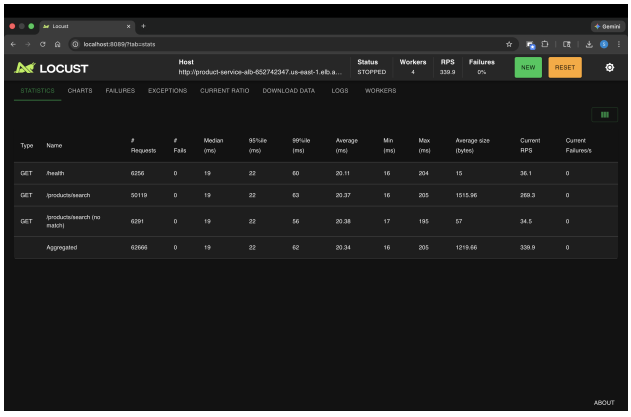
(c) Spike (2–4 instances): 50 users



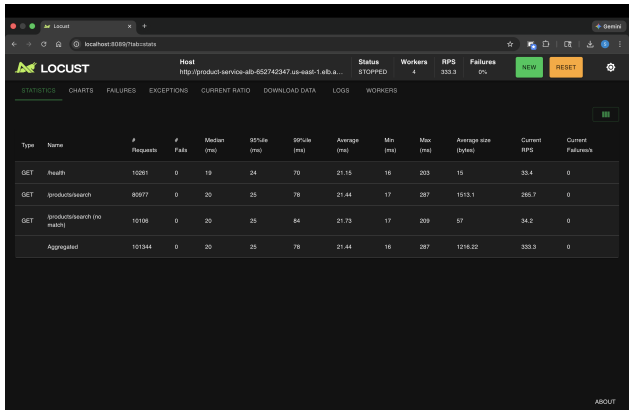
(d) Extreme Spike (2–4 instances): 100 users



(e) Resilience (1 task down): 20 users



(f) Resilience (2 tasks down): 50 users

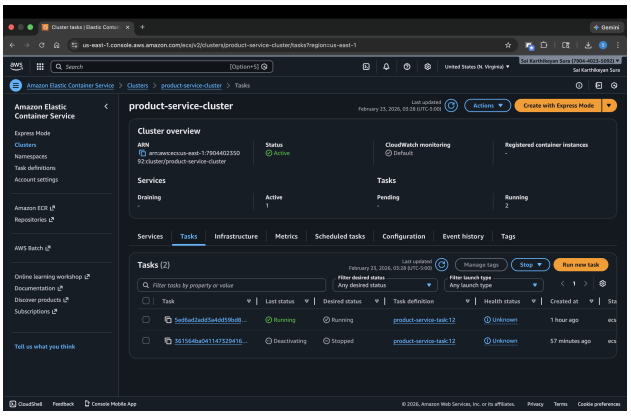


(g) 50% CPU Target Tracking: 20 users

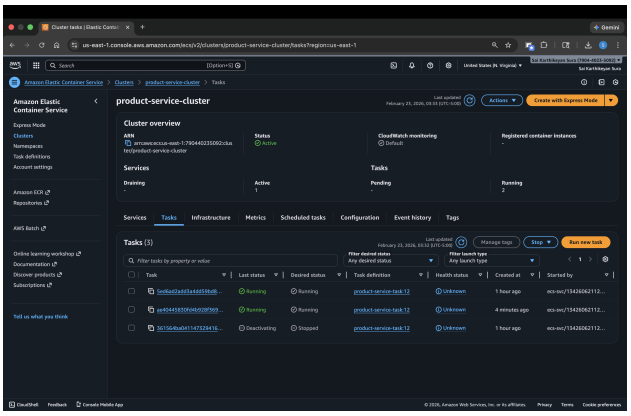
Figure 3: Horizontal scaling experiments.

Resilience Testing

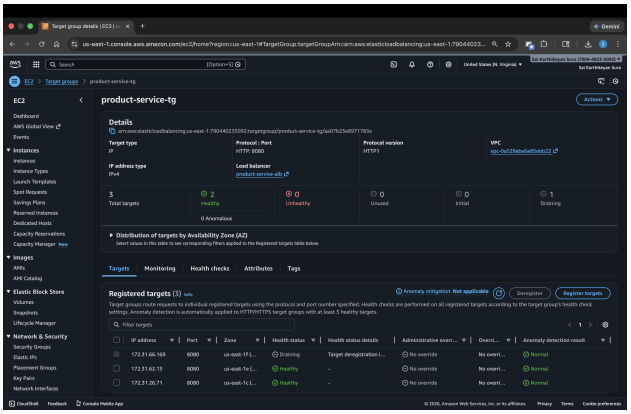
Two resilience tests killed 1 task at 20 users and 2 tasks at 50 users. In both cases, the system maintained 0% failures as the ALB drained connections and ECS automatically replaced the stopped tasks.



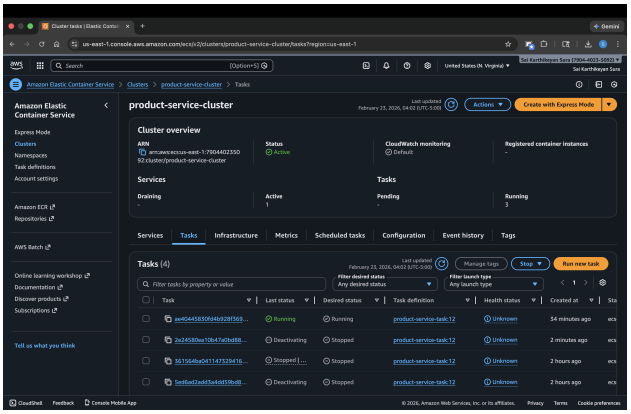
(a) Manual deactivation of 1 task (20 users)



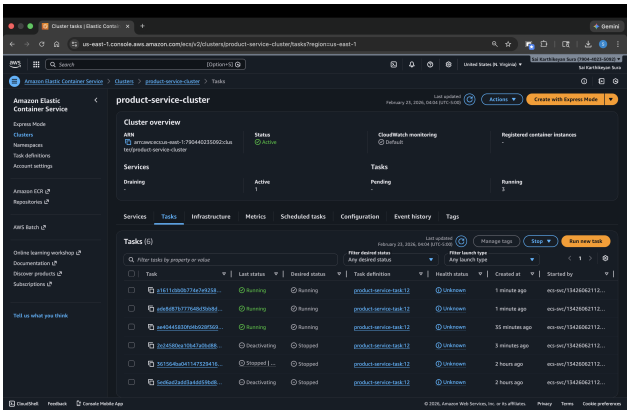
(b) Auto replacement of 1 task



(c) Target group draining during shutdown



(d) Manual deactivation of 2 tasks (50 users)



(e) Auto replacement of 2 tasks under higher load

Figure 4: Seamless task draining and automatic replacement during failures.

Component Roles

Application Load Balancer (ALB) receives HTTP traffic on port 80 and distributes requests across healthy Fargate tasks via round-robin, providing a stable DNS endpoint independent of task count. The **Target Group** performs health checks on `/health` every 30 seconds; tasks

failing three consecutive checks are deregistered and replaced, ensuring traffic is routed only to healthy instances. **Auto Scaling** maintains 70% average CPU utilization by adjusting task count up or down, with a 300-second cooldown for scale-in events.

Monitoring

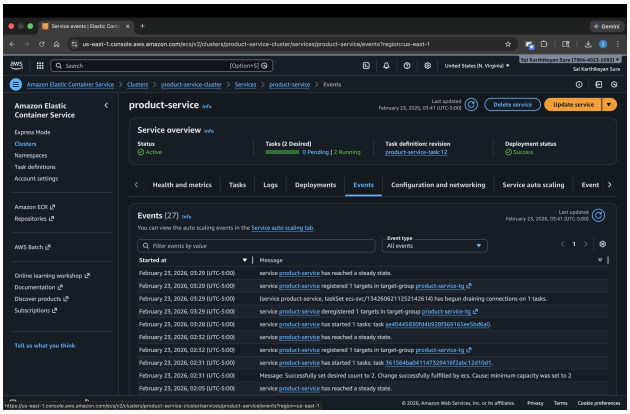
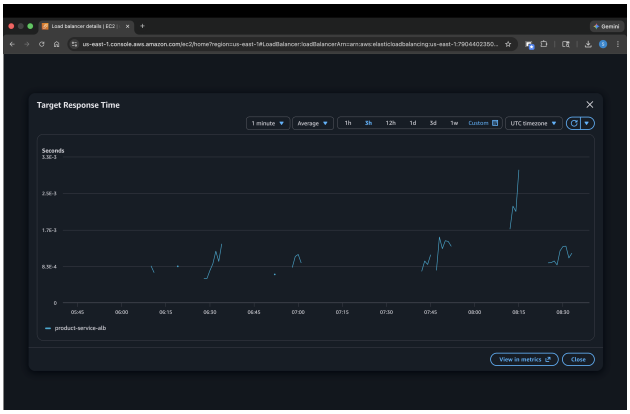


Figure 5: ECS Events log of auto scaling, task registration/deregistration and count changes.

(a) ALB request count per minute



(b) ALB target response time

Figure 6: ALB metrics show request volume scaling with users while latency stays below 3 ms.

Vertical vs. Horizontal Scaling

Dimension	Vertical Scaling	Horizontal Scaling
Approach	Increase CPU/memory per instance	Add more instances behind ALB
Complexity	Simple (change task definition)	Requires ALB, target groups, policies
Fault tolerance	Single point of failure	Survives individual instance failures
Cost curve	Diminishing returns at high tiers	Near-linear cost scaling
Result	256→512 CPU: marginal gain	2–4 instances: 1269 RPS at 100 users

Table 3: Vertical vs. horizontal scaling trade-offs.

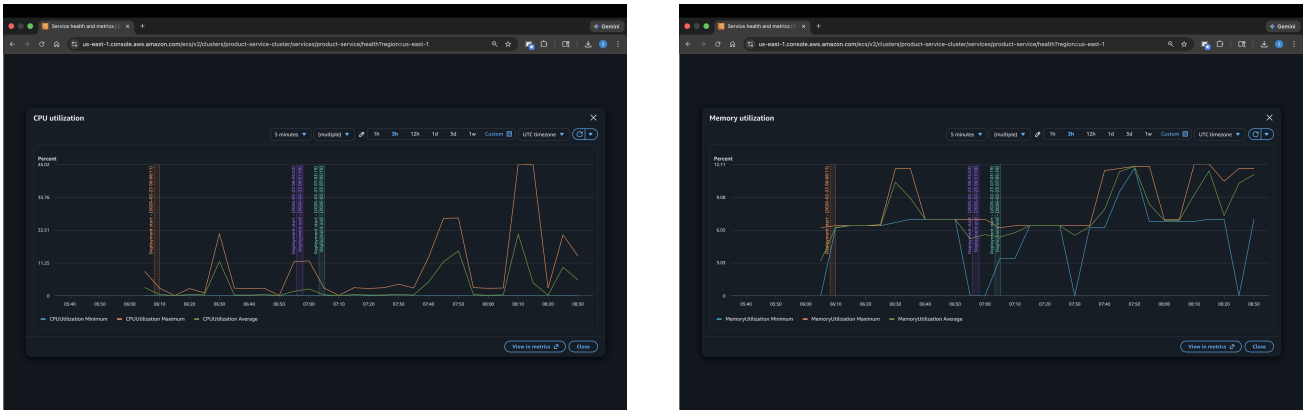
Scaling Behavior Predictions

The system exhibits near-linear throughput scaling: 134 RPS at 20 users, 342 at 50 (stress), 644 at 50 (spike), and 1,269 at 100. Median latency remained at 19–20ms across all tests; only

p95 showed degradation at 100 users (46ms vs. 23ms at 50). The 4-instance configuration could likely handle 200+ concurrent users before requiring a higher maximum task count.
The 50% CPU target policy experiment triggered earlier scaling compared to 70%, providing more headroom at the cost of additional instances. For production workloads with unpredictable spikes, a lower target improves responsiveness; for steady workloads, 70% is more cost-efficient.

CloudWatch Metrics – Full Session

Early peaks (06:10–07:00) correspond to Part II single-instance tests, while sustained activity (07:30–08:30) shows Part III horizontal scaling. The divergence between maximum (orange) and average (green) CPU during Part III confirms the ALB distributes load across instances. Memory remains stable throughout the session.



(a) CPU utilization (b) Memory utilization

Figure 7: Full session CloudWatch metrics.